rg / ripgrep field reference

Dense working notes for people who already use rg. Commands are macOS/POSIX-oriented. Local verification: ripgrep 15.1.0 with PCRE2; jq, fd, and git present. fzf and GNU parallel examples are marked as external tools.

Operating rule

Regex finds candidates. It does not prove compromise, intent, exploitability, or correctness. Treat matches as evidence to inspect, not verdicts.

1. Command Pattern Index

Task	Pattern	Use
Locate matches	`rg -n --column 'auth|token' src/`	Line and column positions for editor jumps and review threads.
List files	`rg -l -0 'PATTERN' | xargs -0 ...`	Machine-safe file list for follow-up tools.
Count	`rg -c 'PATTERN'` `rg --count-matches 'PATTERN'`	File-level line counts vs total match counts.
Extract	`rg -o 'https?://[^[:space:]"\'<>]+'`	Emit just the matched text, then sort, group, or normalize.
Replace preview	`rg -n -r 'NEW' 'OLD'`	Preview replacement output. rg does not edit files.
Audit	`rg -n -i 'secret|token|password' -g '!vendor/**'`	Candidate discovery with scoped exclusions.
Pipe into rg	`ps aux | rg -i 'ssh|vpn|proxy'`	Filter streams where filename context is irrelevant.
Pipe out of rg	`rg -n --color=always 'PATTERN' | less -R`	Keep color in a pager; do not parse colored output in scripts.
Git-aware	`git ls-files -z | xargs -0 rg -n 'PATTERN'`	Search tracked files only, safely handling spaces.
JSON output	`rg --json 'PATTERN' | jq -r 'select(.type=="match")'`	Structured events for scripts and reports. Requires jq.
Multiline / PCRE2	`rg -U -P '[\s\S]{0,200}PATTERN'`	Cross-line candidates, lookaround, lazy quantifiers. Costs more.
Safe scripting	`if rg -q 'BAD'; then exit 1; fi`	Use exit codes intentionally: 0 match, 1 no match, 2 error.

2. Core Command Shapes

Purpose	Command	Notes / footguns
Default search	`rg PATTERN`	Searches paths allowed by ignore rules. No path means current directory.
Scoped path	`rg PATTERN PATH`	Use `--` before paths that may start with `-`.
Editor precision	`rg -n --column PATTERN PATH`	Line and column; useful for quickfix lists and review comments.
Files with matches	`rg -l PATTERN`	One path per matching file. Add `-0` before xargs.
Files without matches	`rg -L PATTERN`	Good for policy checks; beware ignored files.
Count matching lines	`rg -c PATTERN`	Counts lines with one or more matches, not every match.
Count all matches	`rg --count-matches PATTERN`	Multiple matches on one line are counted separately.
Only matched text	`rg -o PATTERN`	Filename prefixes appear when more than one file is searched; add `--no-filename` when needed.
Replacement preview	`rg -r REPLACEMENT PATTERN`	Prints replaced lines. Does not modify files.
Machine output	`rg --json PATTERN`	Structured events; parse with jq or a JSON library.
List searchable files	`rg --files`	Shows files rg would search after ignore/type rules.
Files by glob	`rg --files -g 'GLOB'`	Quote globs so rg, not the shell, interprets them.
Null-safe file pipeline	`rg -0 -l PATTERN | xargs -0 ...`	Required for spaces, tabs, and newlines in path names.

3. Pattern Engineering

Prefer candidate patterns that are specific enough to cut noise, then verify manually. Use `-P` only when you need PCRE2 features.

Needle	Pattern / command	Field note
Word boundary	`rg -n '\btoken\b'`	`\b` avoids matching `tokenizer`.
Anchors	`rg -n '^(export|const)\s+'`	Line-start checks are cheap and precise.
Alternation	`rg -n 'panic|fatal|throw new'`	Order rarely matters in default regex; group when adding quantifiers.
Capture extraction	`rg -o -r '$1' 'id=([0-9a-f-]{8,})'`	`-r` can emit captures without changing files.
Non-capturing group	`rg -P -n '(?:api|access)_?key'`	PCRE2. Useful when capture numbering must stay stable.
Lookahead	`rg -P -n 'password(?=\s*=)'`	PCRE2. Matches `password` only before assignment syntax.
Lookbehind	`rg -P -o '(?<=Bearer )[A-Za-z0-9._~-]+'`	PCRE2. Keep lookbehind fixed-width for portability.
Lazy quantifier	`rg -U -P '<script\b[\s\S]*?</script>'`	PCRE2. Pair with bounds on huge repos when possible.
Unicode class	`rg -n '\p{Letter}+'`	Default rg supports Unicode classes.
IPv4	`rg -n '\b(?:25[0-5]|2[0-4]\d|1?\d?\d)(?:\.(?:25[0-5]|2[0-4]\d|1?\d?\d)){3}\b'`	Candidate IPs; still validate ranges and context.
IPv6 rough	`rg -n '\b(?:[A-Fa-f0-9]{1,4}:){2,7}[A-Fa-f0-9]{1,4}\b'`	Rough net. IPv6 edge cases need a parser.
Email	`rg -n '\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b'`	Good for discovery; not RFC-complete.
URL	`rg -o "https?://[^[:space:]\"'<>]+"`	Double-quote shell string because pattern contains a single quote.
Domain	`rg -io '\b(?:[a-z0-9](?:[a-z0-9-]{0,61}[a-z0-9])?\.)+[a-z]{2,}\b'`	Normalize with lowercase and sort after extraction.
UUID	`rg -n '\b[0-9a-fA-F]{8}-[0-9a-fA-F]{4}-[1-5][0-9a-fA-F]{3}-[89abAB][0-9a-fA-F]{3}-[0-9a-fA-F]{12}\b'`	Version/variant-aware UUID candidate.
SHA-256	`rg -n '\b[A-Fa-f0-9]{64}\b'`	High false positives in generated fixtures.
SHA-1 / MD5	`rg -n '\b[A-Fa-f0-9]{40}\b|\b[A-Fa-f0-9]{32}\b'`	Legacy indicators; weak proof by themselves.
JWT-looking	`rg -n 'eyJ[A-Za-z0-9_-]+\.[A-Za-z0-9_-]+\.[A-Za-z0-9_-]+'`	Header usually starts with `eyJ`, but decode before judging.
PEM private key	`rg -n '-----BEGIN (RSA |DSA |EC |OPENSSH )?PRIVATE KEY-----'`	A private key header is sensitive until proven test-only.
AWS key-looking	`rg -n '\b(AKIA|ASIA)[0-9A-Z]{16}\b'`	Candidate access key IDs; secret half may be elsewhere.
Generic secret assignment	`rg -P -n -i '\b(api[_-]?key|secret|token|password)\b\s*[:=]\s*["'\'']{8,}["'\'']'`	PCRE2 due compact quoting. Review allowlisted test values.
TODO markers	`rg -n '\b(TODO|FIXME|HACK|XXX)\b'`	Useful triage signal in security-sensitive paths.
Download-exec	`rg -n '(curl|wget)[^|;&]*(||;|&&)\s*(sh|bash|zsh|python|perl)'`	Suspicious pattern, not proof. Inspect URL, integrity check, and context.
Eval / exec sinks	`rg -n '\b(eval|exec|Function|child_process\.exec|subprocess\.(Popen|run|call))\b'`	Broad sink search; language-specific follow-up required.
Base64 decode	`rg -n 'base64\s+(-d|--decode)|FromBase64String|atob\(|Buffer\.from\([^)]*,\s*["'\'']base64["'\'']\)'`	Encoded data can be benign; look for decode plus execute.

4. File Selection And Path Control

Control	Example	Notes
Include glob	`rg -n 'PATTERN' -g '*.ts'`	Quote globs so the shell does not expand them first.
Multiple globs	`rg -n 'PATTERN' -g '*.js' -g '*.ts'`	Globs are additive unless negated.
Negative globs	`rg -n 'PATTERN' -g '!{node_modules,dist,build,target,vendor,.venv,__pycache__}/**'`	Brace groups are interpreted by rg when quoted.
Exclude .git	`rg -n 'PATTERN' -g '!.git/**' --hidden`	`--hidden` reveals dotfiles; exclude `.git` explicitly.
Built-in types	`rg -tpy -n 'subprocess'`	Use `rg --type-list` to inspect definitions.
Custom type	`rg --type-add 'infra:*.{tf,yml,yaml,json}' -tinfra -n 'PATTERN'`	Per-invocation type definition; put stable ones in config.
Clear type	`rg --type-clear js --type-add 'js:*.{js,mjs,cjs}' -tjs 'PATTERN'`	Override a built-in type when it is too broad.
Hidden files	`rg --hidden -n 'PATTERN'`	Does not override ignore files by itself.
Ignore override	`rg --no-ignore -n 'PATTERN'`	Searches ignored paths. Expensive in repos with vendor trees.
VCS ignore only	`rg --no-ignore-vcs -n 'PATTERN'`	Ignores `.gitignore` but still honors `.ignore` and `.rgignore`.

Control	Example	Notes
Parent ignores	`rg --no-ignore-parent -n 'PATTERN'`	Useful inside nested repos or inherited noisy worktrees.
Ignore file	`rg --ignore-file audit.ignore -n 'PATTERN'`	Add a policy-specific ignore list without editing repo files.
Follow symlinks	`rg --follow -n 'PATTERN'`	Can walk outside the repo. Pair with `--one-file-system` when appropriate.
Filesystem boundary	`rg --one-file-system -n 'PATTERN' /`	Avoid crossing mounts during broad host scans.
Depth cap	`rg --max-depth 4 -n 'PATTERN'`	Good for home-directory sweeps.
Size cap	`rg --max-filesize 2M -n 'PATTERN'`	Keeps minified blobs and archives from wasting time.
Binary handling	`rg --binary -n 'PATTERN'` `rg --text -n 'PATTERN'`	`--text` can dump garbage; use deliberately.

Ignore hierarchy

`rg` uses explicit flags plus ignore files such as `.ignore`, `.rgignore`, and `.gitignore`. Use `rg --files` before a big audit to prove which files are in scope.

5. Output Shaping

Shape	Command	Why it matters
Line/column	`rg -n --column 'PATTERN'`	Stable coordinates for editors and code review.
Headings	`rg --heading 'PATTERN'` `rg --no-heading 'PATTERN'`	Human scan vs script-friendly repeated path prefixes.
Filename control	`rg --with-filename 'PATTERN'` `rg --no-filename 'PATTERN'`	Stdin and single-file searches change defaults.
Only matches	`rg -o 'PATTERN'`	Use for extraction pipelines.
Replacement	`rg --replace '$1' 'key=([^ ]+)'`	Preview transformation without editing.
Passthru	`rg --passthru 'ERROR|WARN' app.log`	Print nonmatching lines too; useful for highlighted context streams.
Context	`rg -C 2 'PATTERN'` `rg -B 3 -A 5 'PATTERN'`	Use small windows. Large context becomes log dumping.
Trim	`rg --trim -n 'PATTERN'`	Cleaner diffs and reports when indentation is irrelevant.
Max columns	`rg --max-columns 220 --max-columns-preview 'PATTERN'`	Avoid terminal abuse from minified lines while retaining preview.
Color	`rg --color=always 'PATTERN' | less -R`	Only force color for human pagers, not parsers.
Color tuning	`rg --colors 'match:fg:yellow' --colors 'match:style:bold' 'PATTERN'`	Readable highlight themes for long sessions.
JSON	`rg --json 'PATTERN'`	Parse events instead of scraping pretty text.
Vim quickfix	`rg --vimgrep 'PATTERN'`	`file:line:column:match` format for editors.
Null paths	`rg -l -0 'PATTERN'`	Safe handoff to `xargs -0`.
Null data	`rg --null-data 'PATTERN' data.bin`	Treat NUL as line terminator for unusual streams.
Separators	`rg --field-match-separator ' '` `--field-context-separator ' ' 'PATTERN'`	Script-specific separators; prefer JSON for robust parsing.
Path separator	`rg --path-separator / 'PATTERN'`	Normalize path output across platforms.

6. Piping Into rg

When rg reads stdin, it searches stream content and usually loses filename/path context. That is exactly right for process lists and logs, wrong when you meant to search files.

Input stream	Command	Note
Processes	`ps aux | rg -i 'ssh|vpn|proxy'`	Fast local triage; process names can lie.
Git log	`git log --oneline --decorate --all | rg 'fix|security|auth|token'`	Filters rendered history, not file contents.
Homebrew	`brew list --formula | rg 'ssl|crypto|python'`	macOS/Homebrew only.
Docker	`docker ps --format '{{.Names}} {{.Image}}' | rg -i 'db|redis|proxy'`	Requires Docker. Searches Docker output text.
JSON prefilter	`some-json-stream | rg 'token|secret' | jq .`	Only safe when each line remains complete JSON. Usually use jq first.
Log stream	`tail -f app.log | rg --line-buffered 'ERROR|WARN|panic'`	`--line-buffered` keeps live pipelines responsive.
find to files	`find . -type f -name '*.log' -print0 | xargs -0 rg -n 'ERROR|WARN|panic'`	This searches file contents.
fd to files	`fd -0 -e js -e ts | xargs -0 rg -n 'eval|Function\('`	Requires fd. Faster path selection on many systems.
Path names only	`find . -type f -name '*.log' | rg 'auth'`	This searches the file names emitted by find, not the files.
Cat footgun	`cat file.log | rg 'PATTERN'`	Works, but filename context and mmap optimizations are gone.
Direct file	`rg 'PATTERN' file.log`	Prefer this when the input is a file.

7. Piping rg Into Other Commands

Destination	Command	Notes
Pager	`rg -n --color=always PATTERN | less -R`	Human only; color bytes are poison for parsers.
Line count	`rg -n 'PATTERN' | wc -l`	Counts output lines, not necessarily matches. Prefer `--count-matches` for match counts.
Sort unique	`rg -o 'PATTERN' | sort -u`	Good for extracted indicators.
Group counts	`rg -o 'PATTERN' | sort | uniq -c | sort -nr`	macOS/POSIX friendly.
awk fields	`rg --vimgrep 'PATTERN' | awk -F: '{print $1}' | sort -u`	Breaks if paths contain `:`. JSON is safer.
sed normalize	`rg -o "https?://[^[:space:]\"'<>]+" | sed -E 's#https?://([^/:?#]+).*#\1#' | sort -u`	BSD sed `-E` works on macOS.
cut quick	`rg --vimgrep 'PATTERN' | cut -d: -f1-2`	Quick human aid; not robust for colon paths.
jq parse	`rg --json PATTERN | jq -r 'select(.type=="match") | .data.path.text'`	Requires jq. Prefer for automation.
fzf pick	`rg --vimgrep PATTERN | fzf --delimiter : --preview 'sed -n {2}p {1}'`	Requires fzf; not verified locally.
xargs inspect	`rg -l -0 PATTERN | xargs -0 sed -n '1,120p'`	Null-safe path handoff.
Open editor	`rg -l -0 PATTERN | xargs -0 code`	Requires VS Code `code` CLI.
GNU parallel	`rg -l -0 PATTERN | parallel -0 file`	Requires GNU parallel; not verified locally.
Metadata safe	`rg -l -0 'PATTERN' | xargs -0 -I{} sh -c 'printf "%s\n" "$1"; file "$1"' sh {}`	Use `$1` inside `sh -c`; do not interpolate `{}` into shell code.

8. Git-Aware Workflows

Workflow	Command	Field note
Tracked files only	`git ls-files -z | xargs -0 rg -n 'PATTERN'`	Avoids untracked build trash. Empty input behavior differs by xargs implementation.
Changed files	`git diff --name-only -z main...HEAD | xargs -0 rg -n 'PATTERN'`	Searches current working tree versions of files changed vs main.
Staged files	`git diff --cached --name-only -z | xargs -0 rg -n 'SECRET|TOKEN|PASSWORD'`	Pre-commit candidate scan.
Between branches	`git diff --name-only -z origin/main...feature | xargs -0 rg -n 'PATTERN'`	Names from diff; content from working tree unless you check out/read blobs.
History messages	`git log --oneline --decorate --all | rg 'fix|security|auth|token'`	Searches commit metadata text.
History content	`git log --all -G'PATTERN' --oneline -- path/to/file`	Git pickaxe-style search; use git for commit history, rg for worktree files.
git grep	`git grep -n -I -e 'PATTERN' -- '*.ts'`	Tracked files, repo-aware, no xargs. Less flexible than rg for PCRE2/multiline.
rg via git list	`git ls-files -z -- '*.js' '*.ts' | xargs -0 rg -n 'eval|Function\('`	Best blend for tracked scope plus rg speed/features.
CI forbidden pattern	`git diff --cached --name-only -z | xargs -0 rg -n -i 'secret|token|password'`	Wrap exit codes intentionally; see scripting section.
Footgun	`rg -n 'PATTERN' $(git ls-files)`	Breaks on spaces/newlines and can exceed shell arg limits.

9. Security And Forensic Recipes

Defensive candidate discovery only. Matches require context, provenance, and manual review.

Target	Command	Risk lens
Hardcoded credentials	`rg -n -i '\b(password|passwd|pwd|secret|token|api[_-]?key)\b\s*[:=]' -g '!{node_modules,dist,build,vendor}/**'`	Find assignments first; test fixtures and docs need allowlisting.
Token-looking strings	`rg -n 'gh[pousr]_[A-Za-z0-9_]{20,}|eyJ[A-Za-z0-9_-]+\.[A-Za-z0-9_-]+\.[A-Za-z0-9_-]+'`	Candidate GitHub/JWT tokens. Decode or validate only in authorized systems.
PEM private keys	`rg -n '-----BEGIN (RSA |DSA |EC |OPENSSH )?PRIVATE KEY-----'`	Treat as sensitive until proven synthetic.
Download-exec	`rg -n '(curl|wget)[^|;&]*(||;|&&)\s*(sh|bash|zsh|python|perl)'`	Check integrity verification, URL ownership, and install context.
Encoded payload indicators	`rg -n 'base64\s+(-d|--decode)|FromBase64String|atob\(|Buffer\.from\([^)]*,\s*["'\'']base64["'\'']\)'`	Decode plus execute is the stronger signal.
Python shell=True	`rg -U -P -n 'subprocess\.(Popen|run|call|check_output)\([\s\S]{0,300}?shell\s*=\s*True' -g '*.py'`	Look for user-controlled input reaching shell.
Node child_process	`rg -n 'child_process\.(exec|execSync)|\bexecSync?\(' -g '*.{js,ts,mjs,cjs}'`	Follow imports; `execFile`/`spawn` can still be risky with shell options.
SQL concatenation	`rg -P -n '(SELECT|INSERT|UPDATE|DELETE)[^\n]*(\+|\$\{|format\(|%s)'`	Candidate injection. Confirm database API and parameterization.
YAML secrets	`rg -n -i '^\s*(password|secret|token|api[_-]?key):\s*.+' -g '*.{yml,yaml}'`	Kubernetes and CI files deserve priority.
Dockerfile risks	`rg -n 'curl .*| sh|ADD https?://|USER root|chmod 777|--privileged' -g 'Dockerfile*'`	Some are acceptable in builder stages; inspect final image impact.
Cron persistence	`rg -n '@reboot|/etc/cron|crontab|launchctl|systemctl enable'`	Persistence terms in scripts/docs; verify active system state separately.
Shell startup	`rg -n -i 'curl|wget|osascript|launchctl|ssh-agent|alias sudo|function sudo' ~/.zshrc ~/.zprofile ~/.bashrc ~/.bash_profile ~/.profile 2>/dev/null`	Local shell-specific sweep; missing files are normal.
macOS LaunchAgents	`rg -n 'ProgramArguments|RunAtLoad|KeepAlive|StartInterval|WatchPaths' ~/Library/LaunchAgents /Library/LaunchAgents /Library/LaunchDaemons 2>/dev/null`	Presence is not malicious. Check owner, label, path, signature.
npm lifecycle	`rg -n '"(preinstall|install|postinstall|prepare|prepublishOnly)"\s*:' -g 'package.json'`	Package install hooks execute during dependency workflows.
Package hooks	`rg -n 'post-install|pre-commit|prepare|postinstall|build.rs|setup.py'`	Supply-chain review starting points.
Suspicious curl/wget	`rg -n '(curl|wget).*(--insecure|-k|http://|Authorization:|token)'`	Weak TLS, plaintext HTTP, and token-bearing commands need review.
World-writable paths	`rg -n 'chmod\s+777|/tmp/[^[:space:]]+|mktemp\s+-u|/var/tmp'`	Race and privilege-boundary candidates.
SUID mentions	`rg -n 'setuid|suid|chmod\s+[0-7]*4[0-7]{2}|chown\s+root'`	Could be docs or real install logic. Check effective owner/mode separately.

10. Multiline And PCRE2

Use `-U` for multiline search. Use `--multiline-dotall` when you want `.` to cross line breaks, or use `[\s\S]`. Add `-P` for PCRE2 features: lookaround, non-capturing groups, lazy quantifiers, and richer syntax. This is slower and easier to overmatch.

Private key block candidate

```
rg -U -P '-----BEGIN (?:RSA |DSA |EC |OPENSSH )?PRIVATE KEY-----[\s\S]*?-----END (?:RSA |DSA |EC |OPENSSH )?PRIVATE KEY-----'
```

What: Finds full PEM private key blocks across lines.

Shape: `-U` permits cross-line matches; `-P` gives `(?:...)` and lazy `*?`.

Footguns: Can be expensive. Confirm whether the key is test-only, encrypted, committed, and still live.

Approximate JavaScript import plus sink

```
rg -U -P "import\s+.*from\s+[\"']child_process[\"'][\s\S]{0,300}\bexec(?:Sync)?\s*\(" -g '*.{js,ts,mjs,cjs}'
```

What: Finds files where `child_process` import appears near `exec` or `execSync`.

Shape: Bounded `[\s\S]{0,300}` limits overmatch while catching nearby usage.

Footguns: Regex does not parse JavaScript. Aliases, re-exports, comments, and generated code need review.

YAML Secret block candidate

```
rg -U -P '(?m)^apiVersion:\s*v1\s*\n[\s\S]{0,400}^kind:\s*Secret\b' -g '*.{yml,yaml}'
```

What: Finds Kubernetes Secret manifests where fields span lines.

Shape: `(?m)` makes anchors line-oriented inside the multiline match.

Footguns: Base64 in Kubernetes Secret data is encoding, not encryption.

Function block with risky call

```
rg -U -P 'function\s+\w+\s*\([^)]*\)\s*\{[\s\S]{0,1200}\b(eval|Function|exec)\b' -g '*.{js,ts}'
```

What: Approximate scan for risky sinks inside a function body.

Shape: The byte window prevents one match from swallowing a whole file.

Footguns: Nested braces break regex approximations. Use an AST tool for proof.

Config block candidate

```
rg -U -P '(?m)^server:\s*\n(?:\s+.+\n){1,20}\s+ssl:\s*false' -g '*.{yml,yaml}'
```

What: Finds a YAML-ish `server` block containing `ssl: false`.

Shape: Repeats a bounded number of indented lines after `server:`.

Footguns: YAML anchors, comments, and nesting can fool this. Verify with a parser.

Lookaround extraction

```
rg -P -o '(?<=Bearer )[A-Za-z0-9._~-]+'
```

What: Extracts token text after a fixed `Bearer ` prefix.

Shape: Fixed-width lookbehind is the portable PCRE2 shape.

Footguns: Do not print real tokens into shared logs. Redirect carefully.

Multi-line TODO block

```
rg -U -P '(?m)^\s*(//|#)\s*TODO\b[\s\S]{0,500}?(?=^\s*(//|#)?\s*(TODO|FIXME|HACK)\b|\z)'
```

What: Captures a TODO comment and nearby continuation until another marker or EOF.

Shape: Combines multiline mode, a bounded lazy match, and a lookahead stop.

Footguns: Language comment conventions differ. Treat as a triage helper.

HTML script block

```
rg -U -P '<script\b[^>]*>[\s\S]{0,5000}?</script>' -g '*.{html,htm}'
```

What: Finds script blocks without needing a browser parser.

Shape: The 5000-character cap avoids runaway matches in huge pages.

Footguns: HTML parsing with regex is approximate. Use a parser for transformations.

11. JSON Mode And Machine Output

`rg --json` emits event objects. Common types: `begin`, `match`, `context`, `end`, and `summary`. Parse these events; do not scrape colored terminal output.

Goal	Command	Notes
Matching lines	`rg --json 'PATTERN' | jq -r 'select(.type=="match") | .data.lines.text'`	Requires jq. Lines may include trailing newlines.
file:line:match	`rg --json 'PATTERN' | jq -r 'select(.type=="match") | "\(.data.path.text):\(.data.line_number):\(.data.lines.text | rtrimstr("\n"))"'`	Human-readable, less safe than TSV if text contains separators.
TSV report	`rg --json 'PATTERN' | jq -r 'select(.type=="match") | [.data.path.text, .data.line_number, (.data.lines.text|rtrimstr("\n"))] | @tsv'`	Better for spreadsheets and shell post-processing.
Count by file	`rg --json 'PATTERN' | jq -r 'select(.type=="match") | .data.path.text' | sort | uniq -c | sort -nr`	Counts matching lines/events, not submatches.
Submatches	`rg --json 'PATTERN' | jq -r 'select(.type=="match") | .data.submatches[] | [.match.text, .start, .end] | @tsv'`	Offsets are byte offsets within the line text.
Path list	`rg --json 'PATTERN' | jq -r 'select(.type=="match") | .data.path.text' | sort -u`	Use `rg -l -0` for machine-safe path handoff; JSON strings are still line-delimited here.

12. Replacement And Extraction

Replacement boundary

`rg --replace` and `-r` preview modified output. They do not edit files. Use an editor, perl, sd, python, or a purpose-built refactor after review.

Use	Command	Note
Preview literal replacement	`rg -n -r 'NEW' 'OLD'`	Shows what each matching line would look like.
Capture replacement	`rg -n -r '$2, $1' '([A-Za-z]+)\s+([A-Za-z]+)' names.txt`	Capture numbering follows the regex.
Extract URLs	`rg -o "https?://[^[:space:]\"'<>]+" | sort -u`	No filenames if stdin; add `--no-filename` for multi-file extraction.
Extract imports	`rg -o -r '$1' "from [\"'](^[\"']+)[\"']" -g '*.{js,ts,tsx}' | sort | uniq -c | sort -nr`	Requires shell double quotes around the regex.
Normalize domains	`rg --no-filename -o "https?://[^[:space:]\"'<>]+" | sed -E 's#https?://([^/:?#]+).*#\1#' | tr '[:upper:]' '[:lower:]' | sort -u`	BSD sed and POSIX tr friendly.

Use	Command	Note
Duplicate candidates	`rg --no-filename -o "from [\"'][^\"']+[\"']" -r '$1' -g '*.{js,ts}' | sort | uniq -c | sort -nr`	Useful for dependency/import audits.
Refactor preview	`rg -n -r 'logger.$1(' 'console\.(log|warn|error)\(' -g '*.{js,ts}'`	Preview only; use a formatter/refactor tool for edits.

13. Scripting And CI

Case	Pattern	Footgun

Exit codes	`0 = match`
	`1 = no match`
	`2 = error`
		`set -e` treats no-match as failure unless you handle it.

Quiet match	`rg -q 'PATTERN'`	No output; fastest for guards.
Fail on match	`if rg -q 'FORBIDDEN'; then echo 'forbidden pattern'; exit 1; fi`	Use for denylist CI checks.
Fail on no match	`rg -q 'required_symbol' src/ || { echo 'missing required_symbol'; exit 1; }`	Use when a generated artifact must contain a marker.

Handle set -e	`if rg -q 'PATTERN'; then`
	`found=1`
	`else`
	`case $? in 1) found=0 ;; *) exit $? ;; esac`
	`fi`
		Distinguish no match from real errors.

Null-safe paths	`rg -l -0 'PATTERN' | xargs -0 command`	Required for spaces and newlines in filenames.
Avoid command substitution	`# Bad:` `rg -n 'PATTERN' $(git ls-files)`	Word splitting, globbing, and arg limits.

Bash/zsh arrays	`patterns=(-e 'SECRET' -e 'TOKEN')`
	`paths=(src tests)`
	`rg -n "${patterns[@]}" -- "${paths[@]}"`
		macOS zsh and bash support arrays; POSIX sh does not.

Pattern file	`rg -n -f denylist.patterns`	One regex per line. Keep allowlist notes near the pattern file.
Changed-files guard	`git diff --cached --name-only -z | xargs -0 sh -c '[ "$#" -eq 0 ] \|\| rg -n -i "secret\|token\|password" -- "$@"' sh`	Portable empty-input guard; avoids searching cwd by accident.

14. Configuration

ripgrep reads config from the path in `RIPGREP_CONFIG_PATH`; it does not automatically load every project `.ripgreprc` unless your shell or wrapper points to it.

```
cat > .ripgreprc <<'EOF'
--smart-case
--hidden
--glob=!.git/**
--glob=!node_modules/**
--glob=!dist/**
--glob=!build/**
--max-columns=220
--max-columns-preview
--colors=match:fg:yellow
--colors=match:style:bold
--type-add=web:*.{html,css,js,ts,tsx}
EOF
RIPGREP_CONFIG_PATH="$PWD/.ripgreprc" rg -n 'PATTERN'
```

Config move	Example	Risk note
Default smart case	`--smart-case`	Case-insensitive only when pattern has no uppercase.
Hidden with exclusions	`--hidden` `--glob=!.git/**`	Never enable hidden without deciding what to do with `.git`.
Custom types	`--type-add=infra:*.{tf,yml,yaml,json}`	Useful for team policy scans.
Column cap	`--max-columns=220` `--max-columns-preview`	Protect terminals from minified lines.
Project wrapper	`RIPGREP_CONFIG_PATH="$PWD/.ripgreprc" rg "$@"`	A function/script is clearer than global surprise defaults.

Avoid globally	`--no-ignore`
	`--text`
	`--follow`
		Each can explode scope, produce binary noise, or cross trust boundaries.

15. Expert Combo Spells

Changed-file secret scan with null delimiters

```
git diff --name-only -z main...HEAD -- '*.js' '*.ts' '*.py' '*.yml' '*.yaml' |
xargs -0 sh -c '[ "$#" -eq 0 ] || rg -n -i -e "secret|token|password" -e "\b(AKIA|ASIA)[0-9A-Z]{16}\b" -- "$@"' sh
```

What: Scans only changed source/config files for credential candidates.

Shape: Git supplies NUL-delimited paths; the `sh -c` wrapper prevents empty input from becoming an accidental repo-wide search.

Footguns: Patterns are broad. Add allowlists for tests, docs, and known fake values.

PCRE2 multiline private-key JSON report

```
rg -U -P --json '-----BEGIN (?:RSA |DSA |EC |OPENSSH )?PRIVATE KEY-----[\s\S]*?-----END (?:RSA |DSA |EC |OPENSSH )?PRIVATE KEY-----' |
jq -r 'select(.type=="match") | [.data.path.text, .data.line_number] | @tsv'
```

What: Produces a machine-readable path and starting line for private-key candidates.

Shape: JSON avoids scraping colored output; PCRE2 and multiline catch full blocks.

Footguns: Do not dump matched key material into CI logs.

Infra exposure search with custom type

```
rg --type-add 'infra:*.{tf,yml,yaml,json}' -tinfra -n -C 2
'0\.0\.0\.0/0|privileged:\s*true|hostNetwork:\s*true' -g '!vendor/**'
```

What: Finds high-risk infrastructure patterns with two lines of context.

Shape: Custom type keeps the scan focused across Terraform, YAML, and JSON.

Footguns: Context is for triage. Validate semantics with the relevant IaC/Kubernetes parser.

Extract domains from tracked files only

```
git ls-files -z |
xargs -0 rg --no-filename -o "https?://[^[:space:]\"'<>]+" -- |
sed -E 's#https?://([^/:?#]+).*#\1#' |
tr '[:upper:]' '[:lower:]' | sort -u
```

What: Builds a unique domain list from files tracked by git.

Shape: `--no-filename` keeps path prefixes out of the URL stream.

Footguns: URLs in generated fixtures may dominate. Scope with `-g` or git pathspecs.

Shell startup persistence sweep

```
rg -n -i 'curl|wget|osascript|launchctl|ssh-agent|alias sudo|function sudo' ~/.zshrc ~/.zprofile ~/.bashrc
~/.bash_profile ~/.profile 2>/dev/null
```

What: Checks common shell startup files for commands worth inspecting.

Shape: Direct paths keep the scan small and understandable.

Footguns: Missing files are normal. A match is not persistence by itself.

Package lifecycle script audit

```
rg -n '"(preinstall|install|postinstall|prepare|prepack|postpack)"\s*:' -g 'package.json' -C 2
```

What: Finds npm lifecycle scripts that may execute during install or publish.

Shape: Context shows the surrounding package script block.

Footguns: Monorepos may have many package files. Prioritize changed or third-party packages.

Massive repo safe search

```
rg -n 'PATTERN' --max-filesize 2M --max-columns 240 --max-columns-preview -g
'!{.git,node_modules,dist,build,target,vendor,.venv}/**'
```

What: Keeps a broad search responsive in large repositories.

Shape: Size, column, and vendor exclusions reduce pathological files.

Footguns: May miss generated files where the bug actually lives. Remove limits deliberately when needed.

Dangerous sink audit across languages

```
rg -n -P '\b(eval|Function|exec(?:Sync)?|spawn(?:Sync)?|subprocess\.(?:Popen|run|call|check_output))\b' -g
'*.{js,ts,py,sh}'
```

What: Finds execution sinks in JavaScript, TypeScript, Python, and shell files.

Shape: One PCRE2 alternation keeps the sink list reviewable.

Footguns: Sink presence is not vulnerability. Trace data flow from source to sink.

Quick TSV audit report

```
rg --json -i 'secret|token|password' |
jq -r 'select(.type=="match") | [.data.path.text, .data.line_number, (.data.lines.text|rtrimstr("\n"))] | @tsv'
> rg-audit.tsv
```

What: Creates a spreadsheet-friendly report with path, line, and text.

Shape: JSON preserves structure until the final TSV conversion.
Footguns: TSV can still contain sensitive strings. Store and share carefully.

File metadata from match list

```
rg -l -0 'PATTERN' |
xargs -0 -I{} sh -c 'printf "%s\n" "$1"; file "$1"; ls -l "$1"' sh {}
```

What: For each matching file, prints path, file type, and mode/owner summary.
Shape: The shell receives the path as `$1`, avoiding unsafe interpolation.
Footguns: One process per file is slower. Use for investigation, not hot paths.

16. Appendix

Purpose	Flags	Compact note
Scope	`-g, -t, -T, --type-add, --hidden, --no-ignore`	Decide the file set before judging results.
Position	`-n, --column, --vimgrep`	Editor and report coordinates.
Output	`-o, -r, --replace, --passthru, --json`	Extraction, preview, stream shaping.
Context	`-C, -A, -B`	Small context windows beat giant dumps.
Safety	`-0, --null, --, --max-filesize, --max-columns`	Machine-safe paths and terminal protection.
Regex engine	`-P, -U, --multiline-dotall`	Power tools with performance and overmatch costs.

Glob	Meaning	Footgun
`*.py`	Files with `.py` basename in searched directories.	Quote it.
`src/**/*.ts`	TypeScript below `src`.	May be shell-expanded if unquoted.
`!node_modules/**`	Exclude dependency tree.	Order with include globs can matter in complex scopes.
`{js,ts,tsx}`	Brace group handled by rg globset.	Quote so shell does not pre-expand.
`!.git/**`	Exclude git internals when `--hidden` is used.	Essential for repo audits.

Snippet	Use	Caution
`\bword\b`	Whole word.	Boundary behavior follows Unicode mode.
`(?i)secret`	Inline case-insensitive.	Can hide intended case sensitivity.
`[\s\S]{0,500}`	Bounded multiline span.	Needs `-U` to cross lines.
`(?<=Bearer )TOKEN`	Fixed lookbehind.	PCRE2 only.
`.*?`	Lazy match.	PCRE2 only; still can overmatch.

Trouble	Likely cause	Check
No matches	Ignored files, wrong path, shell swallowed pattern.	`rg --files | rg 'expected-file'` `rg --debug PATTERN`
Too many matches	Scope too broad or pattern too generic.	`rg -n PATTERN -g '!vendor/**' -tpy`
Slow scan	Huge files, binary blobs, network mounts.	`rg --max-filesize 2M --one-file-system PATTERN`
Broken script paths	Whitespace/newlines in filenames.	`rg -l -0 PATTERN | xargs -0 ...`
PCRE2 rejected	Missing `-P` or unsupported lookaround.	`rg --version` `rg -P PATTERN`
Colors in parser	`--color=always` leaked into automation.	`rg --color=never --json PATTERN`

Pipeline safety rules

Rule	Use this shape	Avoid this shape
Machine path handoff	`rg -l -0 PATTERN | xargs -0 tool`	`for f in $(rg -l PATTERN); do ...`
Parseable matches	`rg --json PATTERN | jq ...`	`rg --color=always PATTERN | awk ...`
Quoted globs	`rg -g '*.py' PATTERN`	`rg -g *.py PATTERN`
Path separator	`rg -n -- 'PATTERN' "$path"`	`rg PATTERN $path`
No-match in CI	`case $? in 0|1) ;; *) exit $? ;; esac`	`set -e; rg PATTERN`
Empty xargs input	`xargs -0 sh -c '[ "$#" -eq 0 ] \|\| rg PATTERN -- "$@"' sh`	`xargs rg PATTERN`
Sensitive output	`jq -r '[.path,.line] | @tsv'`	`print the whole secret match`
Huge files	`rg --max-filesize 2M --max-columns 240`	`rg --text PATTERN /`

Tool dependency labels

Label	Meaning	Verification note
PCRE2	Requires `rg -P` support.	Local rg has `features:+pcre2`.
jq	JSON event parsing.	Installed locally at build time.
fd	Fast file/path selection before xargs.	Installed locally at build time.
fzf	Interactive fuzzy selection.	Not installed locally; commands require user verification.
git	Tracked/changed/staged file scopes.	Installed locally at build time.
parallel	GNU parallel fan-out.	Not installed locally; commands require user verification.
GNU tools	Options such as GNU-only `xargs -r`.	Avoided in portable examples unless labeled.
Shell-specific	Arrays, redirection, and process wrappers.	Examples target macOS zsh/bash unless noted.

denylist.patterns starter

```
(?i)\b(password|secret|token|api[_-]?key)\b\s*[:=]
\b(AKIA|ASIA)[0-9A-Z]{16}\b
-----BEGIN (RSA |DSA |EC |OPENSSH )?PRIVATE KEY-----
(curl|wget)[^|;&]*(\||;|&&)\s*(sh|bash|zsh|python|perl)
subprocess\.(Popen|run|call|check_output)\([^\n]*shell\s*=\s*True
```

End of reference. Rebuild from `rg_ripgrep_field_reference.py` with the command in `README.md`.